1. University Scholarship Allocation

Problem statement: Evaluate applicants using aggregate marks, subject-wise minimum marks, attendance, and family income. Rank eligible applicants by aggregate marks, then attendance; use lower income as the final tie-breaker.

Algorithm

```
1. Start.
2. Read the number of applicants.
3. For each applicant, read name, aggregate percentage, minimum subject mark, attendance percentage, and annual family incom
4. Mark the applicant eligible only if aggregate >= 80, subject minimum >= 60, attendance >= 75, and income <= 300000.
5. Store eligible applicants and sort by aggregate descending, attendance descending, and income ascending.
6. Display the ranked eligible applicants, or a message if none qualify.
7. Stop.
```

Flowchart

```
[START]
   |
[Read applicant details]
   |
[Check all four eligibility rules]
   |
 <Eligible?> ---- No ----> [Skip applicant]
   | Yes                        |
[Store applicant] <--------------+
   |
[More applicants?] -- Yes --> [Read next applicant]
   | No
[Sort eligible applicants by rules]
   |
[Display ranked list]
   |
 [STOP]
```

Pseudocode

```
BEGIN
  INPUT number_of_applicants
  eligible_list = empty list
  REPEAT for each applicant
    INPUT name, aggregate, subject_min, attendance, income
    IF aggregate >= 80 AND subject_min >= 60 AND attendance >= 75 AND income <= 300000 THEN
      ADD applicant details to eligible_list
    ENDIF
  END REPEAT
  SORT eligible_list by aggregate descending,
       attendance descending, income ascending
  DISPLAY eligible_list
END
```

Python Program

```
applicants = [
    ("Anu", 91, 72, 92, 240000),
    ("Balu", 88, 65, 85, 280000),
    ("Chandu", 91, 70, 92, 200000),
    ("Divya", 79, 80, 90, 150000)
]

eligible = []
for name, aggregate, subject_min, attendance, income in applicants:
    if (aggregate >= 80 and subject_min >= 60
            and attendance >= 75 and income <= 300000):
        eligible.append((name, aggregate, attendance, income))

eligible.sort(key=lambda x: (-x[1], -x[2], x[3]))

print("Scholarship ranking")
if len(eligible) == 0:
    print("No eligible applicants")
else:
    rank = 1
    for applicant in eligible:
        print(rank, applicant[0], "Marks:", applicant[1],
              "Attendance:", applicant[2], "Income:", applicant[3])
        rank += 1
```

Sample Output

```
Scholarship ranking
1 Chandu Marks: 91 Attendance: 92 Income: 200000
```

```
2 Anu Marks: 91 Attendance: 92 Income: 240000
3 Balu Marks: 88 Attendance: 85 Income: 280000
```

Assumptions: Sample rules are assumed for demonstration: aggregate >= 80%, minimum subject mark >= 60%, attendance >= 75%, and annual income <= ■3,00,000.

2. Progressive Electricity Billing

Problem statement: Calculate bills using progressive energy slabs, fixed charge, surcharge, and rebate. Compare monthly bills and flag a large increase.

Algorithm

1. Start and read monthly consumption values.
2. For each month, calculate energy charges using progressive slabs: first 100 units at ■2, next 100 at ■3, and remaining ur
3. Add fixed charge of ■50.
4. Add a 5% surcharge when the energy charge exceeds ■500; otherwise apply a 3% rebate on energy charges.
5. Store and display each bill.
6. Compare the latest bill with the previous bill and flag an increase greater than 25%.
7. Stop.

Flowchart

```
[START]
   |
[Read monthly units]
   |
[Calculate progressive slab charge]
   |
[Add fixed charge]
   |
[Apply surcharge OR rebate]
   |
[Save and display bill]
   |
[More months?] -- Yes --> [Read next month]
   | No
[Compare last two bills]
   |
[Flag increase > 25%]
   |
 [STOP]
```

Pseudocode

```
BEGIN
  INPUT list_of_monthly_units
  FOR each units in list
    IF units <= 100: energy = units * 2
    ELSE IF units <= 200: energy = 100*2 + (units-100)*3
    ELSE: energy = 100*2 + 100*3 + (units-200)*5
    bill = energy + 50
    IF energy > 500: bill = bill + 0.05*energy
    ELSE: bill = bill - 0.03*energy
    DISPLAY bill
  END FOR
  IF latest bill > previous bill * 1.25
    DISPLAY unusual increase warning
  ENDIF
END
```

Python Program

```
monthly_units = [90, 150, 260]

bills = []
for month, units in enumerate(monthly_units, start=1):
    if units <= 100:
        energy = units * 2
    elif units <= 200:
        energy = 100 * 2 + (units - 100) * 3
    else:
        energy = 100 * 2 + 100 * 3 + (units - 200) * 5

    bill = energy + 50
    if energy > 500:
        bill += energy * 0.05
    else:
        bill -= energy * 0.03

    bills.append(bill)
    print("Month", month, "| Units:", units,
          "| Bill: Rs.", round(bill, 2))

if len(bills) >= 2:
    increase = (bills[-1] - bills[-2]) / bills[-2] * 100
    print("Increase from previous month:", round(increase, 2), "%")
    if increase > 25:
```

```
        print("Warning: unusual increase in bill")
    else:
        print("No unusual increase")
```

Sample Output

```
Month 1 | Units: 90 | Bill: Rs. 224.6
Month 2 | Units: 150 | Bill: Rs. 487.0
Month 3 | Units: 260 | Bill: Rs. 1165.0
Increase from previous month: 139.22 %
Warning: unusual increase in bill
```

Assumptions: Illustrative tariff only. Energy slabs are progressive, and surcharge/rebate is calculated on the energy charge.

3. Hospital Billing and Patient Classification

Problem statement: Add consultation, diagnostic, and treatment charges; apply a concession when the patient is a senior citizen or the bill is high; validate payment and classify the account.

Algorithm

1. Start and read patient name, age, consultation fee, diagnostic fee, treatment fee, and amount paid.
2. Reject negative charges or negative payment.
3. Calculate subtotal by adding all three charges.
4. Apply a 10% concession if age is at least 60 or subtotal is at least ■10,000.
5. Calculate final bill and balance due.
6. Classify as Paid, Partially Paid, or Unpaid.
7. Display itemized bill and stop.

Flowchart

```
[START]
   |
[Read patient and charge details]
   |
[Are values valid?] -- No --> [Display invalid input]
   | Yes
[Add charges to get subtotal]
   |
[Age >= 60 OR subtotal >= 10000?]
   | Yes                  | No
[10% concession]     [No concession]
   |                      |
[Calculate final bill and balance]
   |
[Classify payment status]
   |
[Print bill]
   |
 [STOP]
```

Pseudocode

```
BEGIN
  INPUT name, age, consultation, diagnostics, treatment, paid
  IF any charge < 0 OR paid < 0 THEN
    DISPLAY invalid input
  ELSE
    subtotal = consultation + diagnostics + treatment
    IF age >= 60 OR subtotal >= 10000 THEN concession = subtotal * 0.10
    ELSE concession = 0
    final_bill = subtotal - concession
    balance = final_bill - paid
    IF balance <= 0: status = Paid
    ELSE IF paid > 0: status = Partially Paid
    ELSE: status = Unpaid
    DISPLAY itemized bill, balance, status
  ENDIF
END
```

Python Program

```
name = "Ravi"
age = 65
consultation = 800
diagnostics = 2200
treatment = 5000
paid = 4000

if min(consultation, diagnostics, treatment, paid) < 0:
    print("Invalid input: amounts cannot be negative")
else:
    subtotal = consultation + diagnostics + treatment
    if age >= 60 or subtotal >= 10000:
        concession = subtotal * 0.10
    else:
        concession = 0

    final_bill = subtotal - concession
    balance = final_bill - paid

    if balance <= 0:
        status = "Paid"
        balance = 0
    elif paid > 0:
        status = "Partially Paid"
```

```
    else:
        status = "Unpaid"

    print("Patient:", name)
    print("Subtotal: Rs.", round(subtotal, 2))
    print("Concession: Rs.", round(concession, 2))
    print("Final bill: Rs.", round(final_bill, 2))
    print("Amount paid: Rs.", round(paid, 2))
    print("Balance due: Rs.", round(balance, 2))
    print("Payment status:", status)
```

Sample Output

```
Patient: Ravi
Subtotal: Rs. 8000
Concession: Rs. 800.0
Final bill: Rs. 7200.0
Amount paid: Rs. 4000
Balance due: Rs. 3200.0
Payment status: Partially Paid
```

Assumptions: The concession and classification rules are example policies; use the rules supplied by your instructor if they differ.

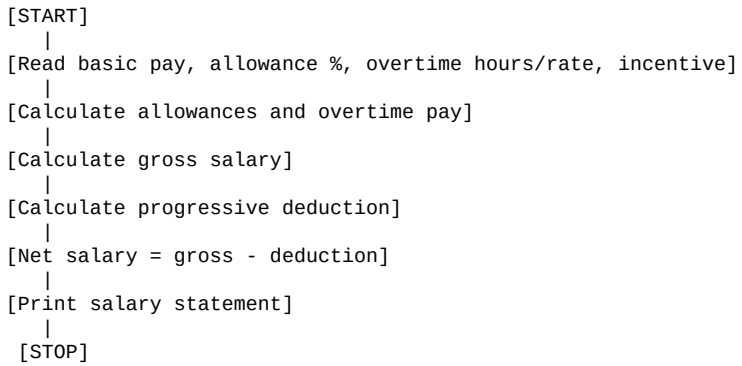
4. Employee Payroll and Deduction Analysis

Problem statement: Calculate gross salary from basic pay, allowances, overtime, and incentives. Apply progressive deductions and print a salary statement.

Algorithm

1. Start and read employee details and salary components.
2. Calculate allowances as a percentage of basic pay, and calculate overtime pay.
3. Calculate gross salary as basic pay plus allowances, overtime, and incentive.
4. Apply progressive deductions: 0% on the first ■10,000, 5% on the next ■10,000, and 10% on the amount above ■20,000.
5. Calculate net salary as gross salary minus deduction.
6. Display the formatted salary statement and stop.

Flowchart



Pseudocode

```
BEGIN
  INPUT employee, basic, allowance_rate, overtime_hours, overtime_rate, incentive
  allowances = basic * allowance_rate / 100
  overtime = overtime_hours * overtime_rate
  gross = basic + allowances + overtime + incentive
  IF gross <= 10000: deduction = 0
  ELSE IF gross <= 20000: deduction = (gross - 10000) * 0.05
  ELSE: deduction = 10000 * 0.05 + (gross - 20000) * 0.10
  net = gross - deduction
  DISPLAY salary statement
END
```

Python Program

```
employee = "Meena"
basic = 25000
allowance_rate = 20
overtime_hours = 8
overtime_rate = 200
incentive = 1500

allowances = basic * allowance_rate / 100
overtime = overtime_hours * overtime_rate
gross = basic + allowances + overtime + incentive

if gross <= 10000:
    deduction = 0
elif gross <= 20000:
    deduction = (gross - 10000) * 0.05
else:
    deduction = 10000 * 0.05 + (gross - 20000) * 0.10

net = gross - deduction

print("------ SALARY STATEMENT ------")
print("Employee:", employee)
print("Basic pay: Rs.", basic)
print("Allowances: Rs.", round(allowances, 2))
print("Overtime pay: Rs.", round(overtime, 2))
print("Incentive: Rs.", incentive)
print("Gross salary: Rs.", round(gross, 2))
print("Deduction: Rs.", round(deduction, 2))
print("Net salary: Rs.", round(net, 2))
```

Sample Output

```
------ SALARY STATEMENT ------
Employee: Meena
```

```
Basic pay: Rs. 25000
Allowances: Rs. 5000.0
Overtime pay: Rs. 1600
Incentive: Rs. 1500
Gross salary: Rs. 33100.0
Deduction: Rs. 1810.0
Net salary: Rs. 31290.0
```

Assumptions: This is a classroom deduction model, not a statutory payroll or tax calculator.

5. Multi-Plan Telecommunications Cost Optimization

Problem statement: Compare mobile plans against usage, validity, and budget. Calculate excess data charges and choose the cheapest plan that meets all constraints.

Algorithm

1. Start and read required data, validity, and budget.
2. Store available plans with name, price, included data, validity, and excess-data rate.
3. For each plan, calculate excess data and total cost.
4. Keep plans that meet required validity and budget.
5. Choose the eligible plan with the lowest total cost.
6. Display plan comparisons and the recommended plan, then stop.

Flowchart

```
[START]
   |
[Read usage, required validity, budget]
   |
[For each available plan]
   |
[Calculate excess data and total cost]
   |
[Meets validity and budget?] -- No --> [Skip plan]
   | Yes
[Keep eligible plan]
   |
[More plans?] -- Yes --> [Next plan]
   | No
[Any eligible plan?] -- No --> [No match]
   | Yes
[Select lowest total cost]
   |
[Display recommendation]
   |
 [STOP]
```

Pseudocode

```
BEGIN
  INPUT data_used, required_days, budget
  eligible_plans = empty list
  FOR each plan
    excess = MAX(0, data_used - plan.included_data)
    total = plan.price + excess * plan.excess_rate
    IF plan.validity >= required_days AND total <= budget THEN
      ADD (plan, total) to eligible_plans
    ENDIF
  END FOR
  IF eligible_plans empty: DISPLAY no eligible plan
  ELSE SORT by total ascending and DISPLAY first plan
END
```

Python Program

```
data_used = 18       # GB required
required_days = 28
budget = 500

# name, price, included GB, validity days, excess charge per GB
plans = [
    ("Basic", 249, 10, 28, 20),
    ("Smart", 399, 20, 28, 15),
    ("Max", 499, 30, 56, 10),
    ("Mini", 199, 8, 14, 25)
]

eligible = []
for name, price, included, validity, rate in plans:
    excess = max(0, data_used - included)
    total = price + excess * rate
    print(name, "- total cost: Rs.", total)
    if validity >= required_days and total <= budget:
        eligible.append((total, name, validity))

if len(eligible) == 0:
    print("No plan satisfies the requirements")
else:
    eligible.sort()
    best = eligible[0]
    print("Best plan:", best[1])
```

```
    print("Total cost: Rs.", best[0])
    print("Validity:", best[2], "days")
```

Sample Output

```
Basic - total cost: Rs. 409
Smart - total cost: Rs. 399
Max - total cost: Rs. 499
Mini - total cost: Rs. 449
Best plan: Smart
Total cost: Rs. 399
Validity: 28 days
```

Assumptions: Usage and plan prices are sample values. Excess data is charged per additional GB in this simplified model.

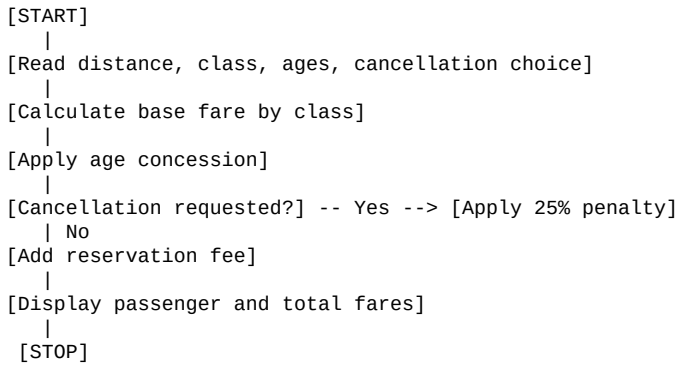
6. Railway Fare and Reservation Calculator

Problem statement: Calculate fares for passengers using distance-based fares, class multipliers, age concessions, reservation fees, and cancellation penalties.

Algorithm

1. Start and read distance, class, passenger ages, and cancellation choice.
2. Calculate base fare per passenger: ■1.50 per km for Sleeper or ■3.00 per km for AC.
3. Apply a 50% concession for children under 12 and a 30% concession for passengers aged 60 or above.
4. Add a reservation fee of ■40 per passenger.
5. If cancelled, deduct a 25% cancellation penalty from the fare before the reservation fee.
6. Display each passenger fare and total, then stop.

Flowchart



Pseudocode

```
BEGIN
  INPUT distance, travel_class, ages, cancelled
  IF travel_class = Sleeper: rate = 1.50
  ELSE: rate = 3.00
  total = 0
  FOR each age
    fare = distance * rate
    IF age < 12: fare = fare * 0.50
    ELSE IF age >= 60: fare = fare * 0.70
    IF cancelled: fare = fare * 0.25
    fare = fare + 40
    DISPLAY fare
    total = total + fare
  END FOR
  DISPLAY total
END
```

Python Program

```
distance = 300
travel_class = "AC"
ages = [35, 10, 62]
cancelled = False

if travel_class == "Sleeper":
    rate = 1.50
else:
    rate = 3.00

total = 0
for number, age in enumerate(ages, start=1):
    fare = distance * rate

    if age < 12:
        fare *= 0.50
    elif age >= 60:
        fare *= 0.70

    if cancelled:
        fare *= 0.25

    fare += 40  # reservation fee
    total += fare
    print("Passenger", number, "| Age:", age,
          "| Fare: Rs.", round(fare, 2))

print("Total fare: Rs.", round(total, 2))
```

Sample Output

```
Passenger 1 | Age: 35 | Fare: Rs. 940.0
Passenger 2 | Age: 10 | Fare: Rs. 490.0
Passenger 3 | Age: 62 | Fare: Rs. 670.0
Total fare: Rs. 2100.0
```

Assumptions: The rates, concessions, reservation fee, and cancellation rule are invented for this exercise and are not official railway tariffs.

7. Retail Billing with Multiple Discount Rules

Problem statement: Create an itemized invoice using product categories, quantities, category-specific tax rates, a threshold discount, and a maximum discount cap.

Algorithm

1. Start and read product name, category, unit price, and quantity for each item.
2. Calculate each line amount and subtotal.
3. Apply a 10% discount if subtotal is at least ■5,000, capped at ■800.
4. Calculate category tax: 5% for groceries, 12% for clothing, and 18% for electronics.
5. Calculate final total as subtotal minus discount plus total tax.
6. Display itemized amounts, discount, tax, and final total; stop.

Flowchart

```
[START]
   |
[Read product lines]
   |
[Calculate line totals and subtotal]
   |
[Subtotal >= Rs. 5000?]
   | Yes -> [10% discount, maximum Rs. 800]
   | No  -> [No discount]
   |
[Calculate tax by category]
   |
[Final = subtotal - discount + tax]
   |
[Print invoice]
   |
 [STOP]
```

Pseudocode

```
BEGIN
  INPUT product list
  subtotal = 0; tax_total = 0
  FOR each product
    line = price * quantity
    subtotal = subtotal + line
    IF category = grocery: tax_rate = 5%
    ELSE IF category = clothing: tax_rate = 12%
    ELSE: tax_rate = 18%
    tax_total = tax_total + line * tax_rate
  END FOR
  IF subtotal >= 5000: discount = MIN(subtotal * 10%, 800)
  ELSE: discount = 0
  final_total = subtotal - discount + tax_total
  DISPLAY itemized invoice
END
```

Python Program

```
products = [
    ("Rice", "grocery", 800, 2),
    ("Shirt", "clothing", 1200, 2),
    ("Headphones", "electronics", 1800, 1)
]

subtotal = 0
tax_total = 0
print("-------- INVOICE --------")

for name, category, price, quantity in products:
    line_total = price * quantity
    if category == "grocery":
        tax_rate = 0.05
    elif category == "clothing":
        tax_rate = 0.12
    else:
        tax_rate = 0.18

    tax = line_total * tax_rate
    subtotal += line_total
    tax_total += tax
    print(name, "x", quantity, "| Amount:", line_total,
          "| Tax:", round(tax, 2))

if subtotal >= 5000:
    discount = min(subtotal * 0.10, 800)
```

```
else:
    discount = 0

final_total = subtotal - discount + tax_total
print("Subtotal: Rs.", round(subtotal, 2))
print("Discount: Rs.", round(discount, 2))
print("Tax total: Rs.", round(tax_total, 2))
print("Final total: Rs.", round(final_total, 2))
```

Sample Output

```
-------- INVOICE --------
Rice x 2 | Amount: 1600 | Tax: 80.0
Shirt x 2 | Amount: 2400 | Tax: 288.0
Headphones x 1 | Amount: 1800 | Tax: 324.0
Subtotal: Rs. 5800
Discount: Rs. 580.0
Tax total: Rs. 692.0
Final total: Rs. 5912.0
```

Assumptions: Category tax rates and discount policy are sample classroom assumptions, not tax advice.

8. Manufacturing Cost and Profitability Analysis

Problem statement: Calculate production cost including materials, labour, wastage, and machine cost. Compare profit per unit with a target.

Algorithm

1. Start and read quantity, material cost per unit, labour cost per unit, wastage percentage, machine cost, and selling pric
2. Calculate material cost and wastage cost.
3. Calculate total production cost by adding material, wastage, labour, and machine costs.
4. Calculate total revenue and profit.
5. Calculate profit per unit and compare it with the target profit per unit.
6. Display cost, revenue, profit, and target status; stop.

Flowchart

```
[START]
   |
[Read production and cost inputs]
   |
[Calculate material and wastage costs]
   |
[Add labour and machine costs]
   |
[Calculate revenue and profit]
   |
[Profit per unit >= target?]
   | Yes              | No
[Target met]      [Target not met]
   |                  |
[Display report]
   |
 [STOP]
```

Pseudocode

```
BEGIN
  INPUT quantity, material_per_unit, labour_per_unit,
        wastage_rate, machine_cost, selling_price, target_profit
  material = quantity * material_per_unit
  wastage = material * wastage_rate / 100
  labour = quantity * labour_per_unit
  total_cost = material + wastage + labour + machine_cost
  revenue = quantity * selling_price
  profit = revenue - total_cost
  profit_per_unit = profit / quantity
  IF profit_per_unit >= target_profit: DISPLAY target met
  ELSE: DISPLAY target not met
  DISPLAY cost, revenue, profit
END
```

Python Program

```
quantity = 500
material_per_unit = 120
labour_per_unit = 35
wastage_rate = 5
machine_cost = 8000
selling_price = 190
target_profit_per_unit = 20

material_cost = quantity * material_per_unit
wastage_cost = material_cost * wastage_rate / 100
labour_cost = quantity * labour_per_unit
total_cost = material_cost + wastage_cost + labour_cost + machine_cost
revenue = quantity * selling_price
profit = revenue - total_cost
profit_per_unit = profit / quantity

print("Material cost: Rs.", round(material_cost, 2))
print("Wastage cost: Rs.", round(wastage_cost, 2))
print("Labour cost: Rs.", round(labour_cost, 2))
print("Machine cost: Rs.", round(machine_cost, 2))
print("Total production cost: Rs.", round(total_cost, 2))
print("Revenue: Rs.", round(revenue, 2))
print("Profit: Rs.", round(profit, 2))
print("Profit per unit: Rs.", round(profit_per_unit, 2))

if profit_per_unit >= target_profit_per_unit:
    print("Profit target achieved")
else:
    print("Profit target not achieved")
```

Sample Output

```
Material cost: Rs. 60000
Wastage cost: Rs. 3000.0
Labour cost: Rs. 17500
Machine cost: Rs. 8000
Total production cost: Rs. 88500.0
Revenue: Rs. 95000
Profit: Rs. 6500.0
Profit per unit: Rs. 13.0
Profit target not achieved
```

Assumptions: The model assumes wastage cost is a percentage of total material cost and allocates all machine cost to this production batch.

9. Multi-Currency Travel Budget Analysis

Problem statement: Convert travel expenses using supplied exchange rates, include a conversion fee, compare spending with a budget, and identify the largest expense category.

Algorithm

1. Start and read budget in INR, exchange rates, and expenses in foreign currencies.
2. For each expense, convert foreign amount to INR using the supplied rate.
3. Add a conversion fee of 2% to each converted expense.
4. Store the converted amount by category and calculate total spending.
5. Find the category with the largest converted expense.
6. Compare total spending with the budget and display the remaining amount or overspend; stop.

Flowchart

```
[START]
   |
[Read budget, exchange rates, expenses]
   |
[Convert each expense to INR]
   |
[Add 2% conversion fee]
   |
[Calculate total and category totals]
   |
[Find largest expense category]
   |
[Total <= budget?] -- Yes --> [Show remaining budget]
   | No --> [Show overspend]
   |
[Display summary]
   |
 [STOP]
```

Pseudocode

```
BEGIN
  INPUT budget_inr, exchange rates, expenses
  total = 0
  FOR each expense (category, amount, currency)
    converted = amount * rate[currency]
    converted = converted * 1.02
    SAVE category and converted amount
    total = total + converted
  END FOR
  largest = category with maximum converted amount
  IF total <= budget: remaining = budget - total
  ELSE: overspend = total - budget
  DISPLAY total, largest category, budget status
END
```

Python Program

```
budget = 60000
rates = {"USD": 83.0, "EUR": 90.0, "INR": 1.0}
expenses = [
    ("Accommodation", 300, "USD"),
    ("Food", 120, "EUR"),
    ("Local transport", 5000, "INR"),
    ("Tickets", 180, "USD")
]

converted_expenses = []
total = 0

for category, amount, currency in expenses:
    inr = amount * rates[currency]
    inr_with_fee = inr * 1.02
    converted_expenses.append((category, inr_with_fee))
    total += inr_with_fee
    print(category, ": Rs.", round(inr_with_fee, 2))

largest = max(converted_expenses, key=lambda x: x[1])
print("Total spending: Rs.", round(total, 2))
print("Largest expense category:", largest[0])

if total <= budget:
    print("Remaining budget: Rs.", round(budget - total, 2))
else:
    print("Budget exceeded by: Rs.", round(total - budget, 2))
```

Sample Output

```
Accommodation : Rs. 25398.0
Food : Rs. 11016.0
Local transport : Rs. 5100.0
Tickets : Rs. 15238.8
Total spending: Rs. 56752.8
Largest expense category: Accommodation
Remaining budget: Rs. 3247.2
```

Assumptions: Exchange rates are fictional sample values. The 2% conversion fee is applied after converting each expense to INR.

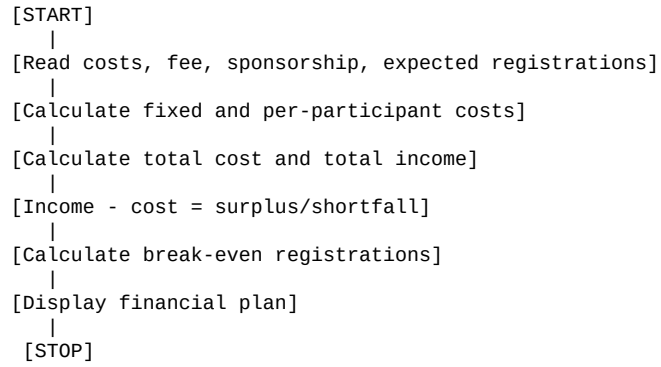
10. Campus Event Financial Planning

Problem statement: Calculate event costs, participant contributions, sponsorship shortfall, and the minimum number of registrations needed to break even.

Algorithm

1. Start and read fixed costs, per-person costs, expected registrations, participant fee, and sponsorship amount.
2. Calculate fixed costs by adding venue, equipment, transport, and certificate setup costs.
3. Calculate variable cost per participant for catering and certificates.
4. Calculate total event cost for expected registrations.
5. Calculate total income from registration fees and sponsorship.
6. Calculate surplus or shortfall.
7. Find the minimum registration count required to cover fixed costs and sponsorship-adjusted costs, then display results an

Flowchart



Pseudocode

```
BEGIN
  INPUT venue, equipment, transport, certificate_setup,
        catering_per_person, certificate_per_person,
        registration_fee, sponsorship, expected_people
  fixed = venue + equipment + transport + certificate_setup
  variable = catering_per_person + certificate_per_person
  total_cost = fixed + variable * expected_people
  income = registration_fee * expected_people + sponsorship
  balance = income - total_cost
  contribution_per_person = registration_fee - variable
  IF contribution_per_person <= 0: break_even = impossible
  ELSE break_even = CEILING((fixed - sponsorship) / contribution_per_person)
  DISPLAY cost, income, balance, break_even
END
```

Python Program

```
venue = 12000
equipment = 5000
transport = 3000
certificate_setup = 1000
catering_per_person = 180
certificate_per_person = 20
registration_fee = 300
sponsorship = 10000
expected_people = 100

fixed_cost = venue + equipment + transport + certificate_setup
variable_per_person = catering_per_person + certificate_per_person
total_cost = fixed_cost + variable_per_person * expected_people
registration_income = registration_fee * expected_people
total_income = registration_income + sponsorship
balance = total_income - total_cost

contribution = registration_fee - variable_per_person
if contribution <= 0:
    break_even = None
else:
    required = (fixed_cost - sponsorship) / contribution
    break_even = int(required)
    if break_even < required:
        break_even += 1
    if break_even < 0:
        break_even = 0

print("Fixed cost: Rs.", fixed_cost)
print("Variable cost per participant: Rs.", variable_per_person)
```

```
print("Total event cost: Rs.", total_cost)
print("Registration income: Rs.", registration_income)
print("Sponsorship: Rs.", sponsorship)
print("Total income: Rs.", total_income)
if balance >= 0:
    print("Surplus: Rs.", balance)
else:
    print("Shortfall: Rs.", -balance)

if break_even is None:
    print("Break-even is not possible with this registration fee")
else:
    print("Minimum registrations to break even:", break_even)
```

Sample Output

```
Fixed cost: Rs. 21000
Variable cost per participant: Rs. 200
Total event cost: Rs. 41000
Registration income: Rs. 30000
Sponsorship: Rs. 10000
Total income: Rs. 40000
Shortfall: Rs. 1000
Minimum registrations to break even: 110
```

Assumptions: Break-even calculation treats sponsorship as reducing fixed costs. It assumes every registration pays the full fee and variable cost is constant per participant.